THE UNIVERSITY OF HONG KONG

School of Computing and Data Science

COMP1110 Computing and Data Science in Everyday Life

Semester 2, 2025-2026

Project Guidelines

Version 1.2 (Feb 27, 2026)

1. Introduction

This group project is a core component of COMP1110: Computing and Data Science in Everyday Life. You will work in a group to select a daily-life problem, model it as a computing and/or data-science problem, and develop a clear and well-justified solution approach. The emphasis is not on advanced programming, but on computational thinking: defining a precise scope, deciding what data and assumptions are needed, designing a simple method to process that data, and explaining the reasoning behind your decisions.

Your work should demonstrate three complementary competencies. First, problem modeling: translating a real-world situation into a structured representation that a computer program can handle. Second, research and evaluation: investigating existing tools or solutions, comparing their strengths and limitations, and using this to motivate your design choices. Third, implementation and evidence: proposing (and partially implementing) a simple solution in Python with text-based interaction and basic file input/output, supported by test inputs and realistic case studies.

Each group will **select one topic from Section 6** and follow the requirements in this document for deliverables. The project is intended to be manageable for students with limited programming experience, while still requiring meaningful teamwork, careful analysis, and clear communication in the project plan and final reports.

The group project counts towards **80% of the final score** of COMP1110.

Students should form their own project groups among classmates. Each group must consist of **4 to 6** students and all members must belong to the same tutorial subclass.

Please note that these guidelines may be updated or amended during the semester. Students should check Moodle regularly for announcements and the latest version of the requirements.

If you have any questions regarding the project, you may post them on the Moodle forum or consult the TAs during tutorials.

2. Project Deliverables

Assessment is based on project deliverables and tutorial attendance (see Section 2.5). This section describes the general expectations for each deliverable; topic-specific requirements are provided in Section 6.

2.1 Project Plan (15%)

The project plan should clearly describe the daily-life problem your group intends to address and define the scope of the project. It should include: (i) a task breakdown and role assignment for all group members; (ii) a brief summary of existing tools/apps related to your topic (detailed analysis and evaluation are **not** required at this stage as they belong to the Group Final Report); and (iii) a reasonable timeline with milestones (for example, a simple Gantt chart).

2.2 Final Code (25%)

You may implement your project in **any programming language**. In your README file, clearly specify the language used, the execution environment, and any compilation or setup instructions if applicable.

We only expect text-based interaction and simple file input/output. Graphical user interfaces and databases are not required (although you may use them). If you want to save data persistently, you may use common file formats such as CSV or JSON.

There is no single required naming convention or coding style; however, the source code should be readable and sufficiently documented for others to understand. Include a README that explains the purpose of each file and how to run your program. Include sample test cases as required by your topic (see Section 6).

Source code should be submitted as a **public GitHub repository** (in addition to the Moodle submission).

2.3 Group Final Report (35%)

The Group Final Report should include an updated task breakdown and role assignment for all group members. This breakdown must be consistent with what is stated in the Individual Final Reports.

The report should explain the daily-life problem, its significance and challenges, and describe how your group modeled it as a computing and/or data-science problem. It should include a detailed survey of existing solutions and a comparison of their approaches. The report should also describe your system design and key functions, supported by case studies. Finally, it

should discuss how you evaluated your work, its limitations, and possible directions for improvement.

2.4 Individual Final Report (10%, Individual Assessment)

Each student must submit an individual report describing their **own contributions** in detail. Your stated contributions must be consistent with the task breakdown and role assignment reported in the Group Final Report.

The individual report should also include: (i) a personal evaluation of the project (e.g., what worked well and what did not), and (ii) reflection on what you learned and how you felt about the project process.

If you used AI tools for coding or writing, you must document this clearly in your individual report, including the name and version of the AI tool used, the prompts you used, and how you modified, verified, or built on the AI-generated outputs.

This report will be assessed **individually**, not on a group basis.

2.5 Group Discussion Participation (15%, Individual Assessment)

All team members are required to attend the final four tutorial sessions (beginning on March 24) and give a **short verbal update** on their project progress to the Student TAs. These progress reports are for monitoring purposes only and will not be graded. The discussion quality will not be evaluated — full marks will be awarded for attendance.

For each session, you will receive half marks if you arrive more than 10 minutes late. You will receive no marks for that session if you arrive more than 20 minutes late.

2.6 Video Demo (0%)

Submit a short video together with your final code. The video should demonstrate the full workflow for at least one of your case studies. Include voice-over or subtitles if needed. The video is for the TAs to follow when reproducing your results and will not be marked.

3 Grading

Project grading will be based on the criteria outlined below. A holistic approach will be used, taking into account factors such as effort, research depth, implementation quality, test data coverage, and overall outcomes. Each group's project will be reviewed by at least two TAs, and a final project grade will be determined and released to every student.

3.1 Project Plan and Reports

- Alignment with the requirements in Section 2 and the topic-specific guidelines in Section 6.
- Structure and organization.
- Formatting and consistency (e.g., headings, spacing, capitalization).
- Professional language and clarity.
- Research depth and comprehensiveness (as appropriate for the topic).
- Appropriateness of evaluation and analysis.
- Precision and conciseness.

3.2 Source Code

- Alignment with the features described in Section 6 for your chosen topic.
- Functionality and completeness of implemented features.
- Data model design.
- Error handling (e.g., missing files, invalid input values).
- Readability and documentation.
- Sample test cases, including clear documentation of the purpose of each test case.
- GitHub repository completeness (e.g., code included, README present, instructions work).

4. Use of AI

Students may use AI for brainstorming, reviewing, paraphrasing, and generating written content or code. However, all use of AI must be clearly documented in the **Individual Final Report**, including the name and version of the AI tool used, the prompts used, and how you modified or verified the AI's outputs.

We understand that not everyone has completed the Python programming course. You are welcome to use AI tools to help generate code (free LLM services are generally sufficient). Be sure to describe your requirements clearly (and include them in your report), carefully review any generated code, and perform thorough testing before relying on it.

5. Timeline for Registration and Submission

Item	**Due Date**
Group registration	March 6 (Fri)
Topic selection Project Plan (15%)	March 23 (Mon) *(You are recommended to submit the plan during the Reading Week, so you can start working early.)*

Final Code (25%) Group Final Report (35%) Individual Final Report (10%) Video Demo (0%)	May 2 (Sat)

You must register your group on Moodle by March 6 (Fri). If you are not yet in a group and need help forming one, select the **"Group by us"** option on Moodle during registration, and the TAs will assign you to a suitable group. To ensure balanced group sizes, the TAs may merge groups with fewer than four members or add additional members to existing groups with fewer than six. Please note that groups failing to register by the deadline will be unable to submit their deliverables and will receive zero marks for the project.

You must choose your project topic on Moodle by March 23 (Mon), at the same time as submitting your Project Plan, after your group members have been finalized.

All deliverables should be submitted to Moodle by 11:59 PM on the due date. For the Final Code, a copy should be submitted as a public GitHub repository, with the link included in the Group Final Report.

6. Project Topics

Students must choose one of the following three topics:

(A) Personal budget and spending assistant;

(B) Smart public transport advisor;

(C) Restaurant queue simulation.

These topics have different focuses.

- Topic A emphasizes research and evaluation of existing solutions.
- Topic B emphasizes implementation and data structures.
- Topic C emphasizes experiments and analysis.

Students are encouraged to read all three topics before making a selection.

Suggested content for each topic is provided in the following pages. You are not required to follow the suggestions exactly; however, they provide a strong starting point and reference. Only deviate from them if you have a better, well-justified approach.

A. Personal Budget and Spending Assistant

Managing personal finances is a common daily-life challenge, especially when spending is frequent and spread across categories such as meals, transport, and subscriptions. In this topic, your group will model everyday spending as a computing problem by defining what information should be recorded for each transaction (e.g., date, amount, category, description) and how those records should be stored and processed to support practical questions such as "Where did my money go?" and "When am I likely to overspend?".

This topic does not aim to build a full-featured commercial budgeting app. Instead, the focus is on (i) clear problem modeling, (ii) research and evaluation of existing budgeting tools, and (iii) a simple, well-justified design that can be partially implemented using basic programming (text-based interaction and simple file input/output). Your research should include a short survey and comparison of existing solutions, as well as a discussion of design trade-offs (at least five) to justify key choices such as manual entry vs automation, few vs many categories, and different ways to set budgets and alerts. The implementation should reflect your model: your program should read transaction data from files, support a text menu for adding/viewing transactions, compute summary statistics (totals by category/period, spending trends), generate rule-based alerts (e.g., daily category caps), and include input validation with sample test cases. You are not required to handle complex forecasting or bank syncing; simple grouping and thresholding is sufficient, as long as your approach is clearly described and consistently applied.

Your final work should include 3–4 case studies that show how your design would be used in realistic spending situations. A case study is a short, concrete scenario (for example: a student capping food spending at HK$50/day, tracking transport for a monthly budget, or detecting subscription creep), supported by specific sample inputs you provide (i.e., transaction files and budget rules). For each case study, you should run your program on the sample data, present the summaries and alerts produced, and discuss what the system does well, where it may fail (e.g., miscategorized expenses, irregular one-offs), and how an existing tool would handle the same scenario. Finally, discuss the strengths and limitations of your solution, and directions for future improvement.

A.1 Suggested content for Personal Budget and Spending Assistant

Area	**Task**	**What to produce**
Research	Survey budgeting tools	Shortlist 4-6 popular budgeting apps/spreadsheets. Identify target users, key features (categorization, alerts, reports), missing features, and assumptions for each.
Research	Compare solutions	Create a table comparing tools across 5-6 criteria: data entry method, category flexibility, budget rule types, reporting granularity, ease-of-use, privacy. Write 1-2 paragraphs summarizing key takeaways.
Research	Design trade-offs	Discuss at least 5 specific trade-offs with pros/cons, e.g., manual entry vs bank sync, few vs many categories, daily vs weekly budgets, local files vs cloud, simple totals vs trend predictions.
Code	Data model	Define transaction schema (date, amount, category, description, optional notes). Define budget rules (category, time period, threshold, alert type). Create category list configuration.
Code	File I/O	Load/save transactions and budget rules from simple text files. Handle missing files, empty files, malformed data gracefully with clear error messages.
Code	Menu interface	Text menu with options: add transaction, view all/filter by date/category, summaries, alerts, load/save data, configure budgets. Include input validation (dates, numbers, categories).
Code	Summary statistics	Compute: total spending, per-category totals, time-based totals (daily/weekly/monthly), top-3 categories, spending trends (last 7/30 days).
Code	Rule-based alerts	Implement 4-5 alert rules, e.g., daily/weekly category caps, percentage thresholds (e.g., transport >30% total), consecutive overspend days, uncategorized transaction warnings.
Code	Test data generator	Create a function to generate realistic test transaction sets (varying dates, amounts, categories) for testing summaries and alerts. Include edge cases (zero spending, all uncategorized).
Case studies	Scenario design	Create 3-4 realistic scenarios with specific goals and sample input files, e.g. daily food budget HK$50, monthly transport tracking, subscription creep detection. Provide exact transaction files + budget rules for each.
Case studies	Evaluation discussion	For each scenario, run your program on sample data, show summaries/alerts output, discuss what works well (clear overspend warnings), limitations (manual categorization errors, irregular one-offs), comparison to commercial tools. Discuss the strengths and limitations of your solution, and suggest directions for future improvement.

B. Smart Public Transport Advisor

Choosing how to travel in a city is a daily decision that involves trade-offs between time, cost, convenience, and the number of stops. In this topic, your group will model everyday commuting as a computing problem by defining a small transport network made up of stops and segments, where each segment connects exactly two stops and represents one way of travelling between them (for example, an MTR ride, a bus leg, or a walking segment). Each segment has properties such as duration and cost, and your task is to use this network to construct journeys (sequences of segments) between a chosen origin and destination.

This topic does not aim to reproduce a full-featured mapping or route-planning service. Instead, the focus is on (i) clear problem modeling, (ii) research and evaluation of existing journey-planning tools, and (iii) a simple, well-justified design that can be partially implemented using basic programming (text-based interaction and simple file input/output). Your research should include a short survey and comparison of existing tools (for example, what inputs they require and what outputs they provide, such as ETAs, fares, and transfers). The implementation should reflect your model: your program should read a hand-crafted transport network (stops and segments with duration and cost), accept user inputs for origin, destination, and preference (e.g. “cheapest”, “fastest”, “fewest segments”), generate some possible journeys as sequences of segments, and then rank those journeys according to the selected preference. You are not required to find mathematically optimal journeys (which would need more advanced graph algorithms); using simple methods to find reasonable candidate journeys is sufficient, as long as your approach is clearly described and consistently implemented. The program should include basic input validation and sample test cases.

Your final work should include 3–4 case studies that show how your design would be used in realistic travel situations. A case study is a short, concrete scenario (for example: a student commuting to campus on a tight budget, a user who wants the fastest route even if it costs more, or a user who prefers routes with fewer segments), supported by specific sample inputs you provide (i.e. a particular transport network, origin/destination pairs, and preference settings). For each case study, you should run your program on the sample data, present the journeys your system generates and their ranking, and discuss what the system does well, where it may fail (for example, missing a clearly better route because your method is simple, or being sensitive to how the network was defined), and how an existing journey-planning tool would handle the same scenario. Finally, discuss the strengths and limitations of your solution, and direction for future improvement.

B.1 Suggested content for Smart Public Transport Advisor

Area	**Task**	**What to produce**
Research	Survey existing solutions and compare	Shortlist some existing tools (e.g., Google Maps, Citymapper, MTR Mobile, HKBUS.APP) and identify their target users, key features, missing features, assumptions. Using a table, compare the above tools across criteria (ETA, fare display, transfer handling, accessibility info, offline use). Write a paragraph of takeaways: what design ideas you find useful or problematic.
Research	Problem framing	Define core entities and concepts such as “stop”, “segment”, “cost”, “duration”, “journey”, and “preference mode”. List what inputs the program will take and what outputs it will produce. State what the program will not do (e.g. no live maps).
Code	Network data model	Define how you represent stops and segments. Create one or more small, hand-crafted (or code-generated) networks (minimum: 10 stops and 20 segments) that are rich enough to support your case studies.
Code	Menu interface	Implement a text-based menu with options such as: list all stops, query journeys, show network summary, load networks, exit.
Code	Input validation	Handle invalid or missing inputs, e.g., unknown stops, same origin and destination, invalid preference mode.
Code	File I/O	Load the transport network from a simple text file. Handle missing file and empty file gracefully. Document the format in your README.
Code	Candidate journey generation	Implement a simple method to generate journeys between a given origin and destination, such as: ● Randomly moving to an adjacent stop until reaching the destination, or ● Depth-limited search up to a certain number of segments. You don’t need to guarantee optimal routes.
Code	Scoring and ranking	For each candidate journey, compute statistics such as total duration, total cost, number of segments. Implement several preference modes, such as “cheapest”, “fastest”, “fewest segments”. Output the top few journeys for each query, with a breakdown of cost/time/segments.
Case studies	Scenario design	Write 3–4 user scenarios with goals, e.g., “budget commuter”, “last-minute student”, “transfer-averse user”. For each scenario, specify which network layout you are using, the origin and destination stops, and the preference mode(s) to test.
Case studies	Evaluation discussion	For each case, provide a sample input (network + origin/destination + preference), run your program, and record the output. Describe what worked and what did not. Briefly discuss how an existing tool might handle the same scenario differently. Discuss the strengths and limitations of your solution, and suggest directions for future improvement.

C. Restaurant Queue Simulation

Choosing how to manage restaurant queues is a daily challenge that involves trade-offs between waiting time, table utilization, and group size matching. In this topic, your group will model restaurant operations as a computing problem by defining customer arrival scenarios (lists of groups where each has group size, arrival time, and dining duration) and restaurant settings (tables with seat capacities, plus multiple queues where each queue serves a specific group size range like 1–2, 3–4, or 5+ people using first-come-first-served). Your task is to use these inputs to run a simulation that assigns arriving groups to matching queues and seats groups at suitable empty tables (no table sharing for simplicity).

This topic does not aim to reproduce a full-featured restaurant management service. Instead, the focus is on (i) clear problem modeling, (ii) research and evaluation of existing queue management approaches, and (iii) a simple, well-justified design that can be partially implemented using basic programming (text-based interaction and simple file input/output). Your research should include a short survey and comparison of existing approaches (for example, single queue vs. size-based queues, VIP priority queues, and their pros/cons for different restaurant types). The implementation should reflect your model: your program should read hand-crafted restaurant settings (tables and queues) and customer arrival scenarios from files, simulate arrivals by assigning groups to size-matching queues, seat the earliest-waiting suitable group whenever a table frees up, track dining durations, and output metrics like average wait time, max queue length, and table utilization. You are not required to use advanced event queues or real-time processing; a simple step-by-step or event-based simulation is sufficient, as long as your approach is clearly described and consistently implemented. The program should include basic input validation.

Your final work should include 3–4 case studies that show how your design reveals the effects of changing restaurant settings in realistic scenarios. A case study is a short, concrete scenario (for example: a small café during lunch rush using a lot of small tables and a single queue versus the same café with larger but fewer tables and multiple queues; or a dim sum hall comparing coarse queues like 1–6 vs. 7+ versus fine-grained queues), supported by specific sample inputs you provide (i.e. particular restaurant settings and customer arrival scenarios). For each case study, you should run your program on the sample data, present the simulation outputs (e.g., wait times, groups served, queue peaks, table utilization) and seating decisions for each setting variation, and discuss the trade-offs revealed (e.g., larger tables reduce queue peaks but waste space for small groups; single queue simplifies operations but delays large parties), what the system does well, where it may fail, and how real restaurants balance these choices. Finally, discuss the strengths and limitations of your solution, and directions for future improvement.

C.1 Suggested content for Restaurant Queue Simulation

Area	**Task**	**What to produce**
Research	Survey queue approaches	Shortlist real-world queue management strategies (e.g., single queue, size-based queues like 1-2/3-4/5+, priority systems) used in restaurants/cafés/dim sum halls. Identify target scenarios, key benefits, drawbacks, and assumptions for each.
Research	Compare approaches	Create a table comparing strategies across criteria like average wait time, table utilization, fairness to group sizes, peak-hour performance, operational complexity. Include a short takeaways paragraph.
Research	Document assumptions	List modeling assumptions clearly (e.g., strict first-come-first-served per queue, no table sharing, group-size matching).
Code	Data model	Define structures for customer groups (group size, arrival time, dining duration), tables (capacity, current status), queues (group size range min-max, list of waiting groups).
Code	File I/O	Load restaurant settings and customer arrival scenarios from simple text files. Handle missing/empty files gracefully.
Code	Core simulation	Implement event-based or step-by-step simulation: process arrivals by assigning to matching queue; when table frees, seat earliest suitable group from any queue; advance time tracking dining ends.
Code	Metrics computation	Calculate and output statistics, e.g., average/max wait time, max queue length per queue, groups served, table utilization (% time occupied), service level (% groups seated within X minutes).
Case studies	Scenario design	Create 5-6 paired scenarios where each pair varies exactly one factor (e.g., table sizes, number of queues, queue granularity) to reveal clear trade-offs, using the same customer arrival pattern across both variations within a pair. For each scenario and variation, provide exact input files (restaurant settings + customer arrivals) with the required format.
Case studies	Evaluation analysis	For each scenario pair, run your simulation on both settings and present metrics side-by-side (wait times, utilization), analyze trade-offs (e.g., large tables reduce queues but waste space for small groups), discuss limitations (e.g., no customer walkaways), compare to real-world practices, and suggest directions for improving simulation accuracy and usefulness.